

RELATÓRIO – RA3

Tabela Hash por Encadeamento Separado

Aluno: Gabriel Calado da Silva Castro

Disciplina: Resolução de Problemas Estruturados em Computação

Trabalho: Recuperação RA3

1. Introdução

Este trabalho implementa uma Tabela Hash com **encadeamento separado** em Java básico.

O objetivo é analisar o desempenho de três funções hash:

- **H_DIV** → divisão
- **H_MUL** → multiplicação
- **H_FOLD** → dobramento

A análise compara colisões, tempo de inserção, custo das listas e desempenho nas buscas (hits e misses), considerando diferentes tamanhos de tabela e datasets.

2. Metodologia

“Distribuições mais uniformes reduzem o custo médio no encadeamento separado.”

2.1 Implementação

A tabela hash foi implementada usando:

- Vetor de listas encadeadas simples (No)
- Registros contendo apenas uma chave inteira de 9 dígitos
- Sem uso de:

- HashMap, ArrayList, Math, StringBuilder
- exceções, operadores ternários ou métodos avançados

Cada operação utiliza apenas laços com contadores e tipos primitivos.

2.2 Funções Hash Avaliadas

H_DIV

$$h = k \% m$$

H_MUL

Baseada na constante de Knuth:

$$h = ((k * A) \ggg 28) \% m$$

H_FOLD

Soma blocos decimais de 3 dígitos da chave:

$$\text{ex: } 123456789 \rightarrow 123 + 456 + 789$$

2.3 Tamanhos da tabela (m)

Definidos pelo professor:

- 1009
- 10007
- 100003

Todos são primos, reduzindo padrões de aglomeração.

2.4 Conjuntos de dados (n)

- 1000
- 10000
- 100000

As chaves são geradas como inteiros aleatórios de 9 dígitos usando:
`Random(seed)`, com seeds:

- 137
- 271828
- 314159

Garantindo total reprodutibilidade entre execuções.

2.5 Inserções

Para cada combinação (m, n, função, seed):

- Tabela iniciada vazia
- Todos os elementos são inseridos
- São registradas:
 - **coll_tbl**: colisões na tabela
 - **coll_lst**: colisões de lista
 - **ins_ms**: tempo médio (5 repetições)

A contagem de colisões segue a regra:

- compartimento vazio → 0 colisões
- compartimento ocupado → 1 colisão de tabela
- passos dentro da lista → colisões de lista

2.6 Buscas

O lote de busca contém:

- 50% chaves presentes
- 50% chaves ausentes

- ordem embaralhada

Cada experimento coleta:

- **find_ms_hits** – tempo dos hits
- **find_ms_misses** – tempo dos misses
- **cmp_hits** – comparações até achar
- **cmp_misses** – comparações até concluir ausência

2.7 Auditoria

Antes de cada experimento o programa imprime:

```
H_DIV 1009 137  
checksum XXXX
```

O checksum é calculado com a soma dos 10 primeiros $h(k)$ mod 1.000.003. Isso garante autenticidade e reprodutibilidade.

3. Resultados e Análise

3.1 Auditoria e checksum

Para cada experimento o programa imprime uma etiqueta de auditoria antes das inserções, por exemplo:

```
H_DIV 1009 137  
checksum 4960
```

Os checksum foram calculados somando os primeiros 10 valores $h(k)$ (na ordem de geração) e aplicando mod 1.000.003. Essas linhas confirmam a reprodutibilidade das execuções e aparecem no stderr, não poluindo o CSV.

3.2 Visão geral dos resultados

Os resultados completos estão em `resultados.csv`. A seguir destacam-se padrões observados e exemplos numéricos representativos extraídos desse arquivo.

Tendências gerais observadas

- O tamanho da tabela m afeta fortemente as colisões: tabelas pequenas (por exemplo $m = 1009$) produzem muito mais colisões e listas muito maiores que tabelas grandes ($m = 100003$).
- Entre as funções hash, **H_MUL** e **H_DIV** apresentam comportamento similar e geralmente melhor distribuição do que **H_FOLD**.
- **H_FOLD** tende a concentrar chaves em determinadas posições (soma de blocos decimal), causando grande aumento nas colisões de lista em cenários com n grande.

3.3 Exemplos numéricos representativos

(Abaixo alguns trechos do `resultados.csv` que ilustram os padrões. Colunas: `m,n,func,seed,ins_ms,coll_tbl,coll_lst,find_ms_hits,find_ms_misses,cmp_hits,cmp_misses,checksum`)

- **Caso** $m = 100003, n = 100000$ (comparação entre funções):
 - `100003,100000,H_DIV,137,3,36705,13112,0,0,500,482,453407`
 - `100003,100000,H_MUL,137,3,36818,13278,0,0,500,470,571157`
 - `100003,100000,H_FOLD,137,51,97342,2746197,0,0,588,27991,18076`

Interpretação: H_DIV e H_MUL têm números semelhantes de colisões na tabela (~36k) e colisões de lista ~13k. H_FOLD tem colisões na tabela e, principalmente, colisões de lista **muito maiores** (~2.7M), indicando péssima distribuição para esse caso.
- **Caso** $m = 1009, n = 100000$ (tabela pequena, fator de carga alto):
 - `1009,100000,H_DIV,137,119,98991,4861946,0,0,633,49723,4960`
 - `1009,100000,H_MUL,137,91,98991,4857647,0,0,618,49446,6725`
 - `1009,100000,H_FOLD,137,73,98991,4853969,0,0,639,49524,5968`

Interpretação: Com $m=1009$ praticamente todo n gera colisão na tabela (`coll_tbl` ≈ 98991), e as colisões de lista tornam-se enormes (milhões). Nessas condições o custo das buscas cresce muito; as diferenças entre funções ficam ofuscadas pelo fator de carga elevado.
- **Caso** $m = 10007, n = 1000$ (tabela grande em relação a n):

- 10007,1000,H_DIV,137,0,49,0,0,0,510,44,58248
 - 10007,1000,H_FOLD,137,0,248,61,0,0,588,292,18076
- Interpretação:** Para n pequeno, H_DIV e H_MUL produzem baixas colisões; H_FOLD pode ainda concentrar alguns casos ruins (ex.: col_tbl 248 e col_lst 61), dependendo da seed.

3.4 Tempos de inserção e resolução da granularidade temporal

- Os tempos de inserção (ins_ms) aparecem, em muitos casos, como **valores muito pequenos** (0–10 ms) para datasets pequenos e tabelas grandes; para $n = 100000$ e m reduzido os tempos sobem para dezenas ou centenas de ms.
- Observação importante: os tempos de busca por hit/miss no CSV aparecem como 0 em muitas linhas — isso ocorre porque o tempo por lote (soma dos tempos de tamLote) arredondado para milissegundos ficou pequeno ou a parcela de hits/misses foi medida em sub-milissegundos; por isso **as métricas de comparações (cmp_hits, cmp_misses) são mais informativas** para analisar custo da busca em vez dos tempos isolados.

3.5 Hits vs Misses (comparações)

- cmp_hits e cmp_misses mostram comportamento esperado:
 - Para $m=100003$, $n=100000$ os cmp_hits ficam em torno de 500 e cmp_misses próximos (~480), refletindo listas moderadas.
 - Para $m=1009$, $n=100000$ cmp_misses chegam a ~49723, mostrando que misses percorrem quase toda a lista em cenários de alto fator de carga.
- Conclusão: **misses são muito mais caros** que hits em termos de comparações quando o fator de carga é alto.

3.6 Comparação entre funções hash

- **H_MUL:** geralmente melhor ou equivalente a H_DIV em distribuição, com col_tbl e col_lst próximos ou ligeiramente menores. Ex.: para $m=100003$, $n=100000$ H_MUL tem col_lst ≈ 13278 contra H_DIV ≈ 13112 .
- **H_DIV:** bom comportamento especialmente com m primo (como requisitado), resultados estáveis.
- **H_FOLD:** frequentemente o pior; em cenários de $n=100000$ apresentou col_lst milhares de vezes maior que as outras funções, indicando que dobramento decimal não distribuiu bem as chaves geradas.

4. Conclusão

- Tabelas com m maior (100003) apresentam distribuição e desempenho muito melhores.
- **H_MUL** e **H_DIV** mostraram distribuição adequada e menor custo médio; **H_MUL** frequentemente tem leve vantagem.
- **H_FOLD** apresentou, em vários cenários de maior n , um número muito elevado de colisões de lista, tornando-o a pior escolha entre as três.
- Misses são sensivelmente mais caros que hits quando α é grande.